Update - Differential Analysis of LLMs in Text2SQL Generation

Jaykumar Patel
patel.jay4802@utexas.edu

An-Jung Huang
ajhuang@utexas.edu

Ryan Kolm
ryankolm@utexas.edu

Abstract

Large Language Models (LLMs) have shown remarkable capabilities in translating natural language questions into executable SQL queries, a task known as Text2SQL. In this work, we conduct a systematic evaluation of several state-of-the-art LLMs and two distinct Text2SQL frameworks: agentless and agent-based pipelines. Our experiments assess accuracy, consistency, and execution robustness on the BIRD-SQL Dev Set. By enabling users to query structured databases through natural language, effective Text2SQL systems have the potential to democratize data access, allowing non-developers to retrieve complex information from structured databases without needing specialized technical skills. Our study aims to inform future design choices for building more reliable and accessible natural language interfaces to databases.

1 Introduction

As large language models (LLMs) continue to envolve and improve, an interesting application of LLMs is in the generation of SQL queries from natural language. This is known as Text2SQL. Text2SQL can enable non-experts to extract valuable information from a database without having to learn complex query languages like SQL. By simply describing the data they want in plain English, users can leverage LLMs to translate their requests into structured queries that interact directly with a database. This has the potential to democratize data access across organizations, reduce reliance on technical teams for routine data retrieval tasks, and improve overall efficiency in data-driven decision-making.

In this project, we compare the performance of different LLMs in generating SQL queries from natural language. We explore 2 different frameworks, Agentless and Agent-based, and 3 different LLMs, Claude 3.5 Sonnet, GPT-4o mini, and Llama3.1 8B. In the Agentless framework, the descriptions of all tables in the database (relevant and irrelevant) are passed as context to the LLM when generting the SQL query given the natural language query. In the Agent-based framework, there are 3 agents that handle specific tasks such as determining the number of relevant tables, retrieving the descriptions of the relevant tables, and generating a SQL query given a natural language query. We run our Agentless and Agent-based frameworks with the 3 different LLMs on the BIRD-SQL Dev Set benchmark.

By systematically analyzing various LLMs and frameworks, we aim to understand how model choice and pipeline design impact the accuracy, consistency, and execution success of generated SQL queries. Our findings can help guide the development of more reliable and accessible Text2SQL systems, ultimately making database interaction easier and more intuitive for a broader range of users.

2 Previous Work

Text2SQL has been an active area of research, with approaches evolving alongside advances in natural language processing. Early methods primarily relied on rule-based systems [**?**] and semantic parsing [**?**], which required significant domain-specific engineering and struggled to generalize across different database schemas. With the rise of deep learning, encoder-decoder architectures and pre-trained language models began to dominate, enabling more flexible mappings from natural language to SQL queries.

Recent work has explored building Text2SQL pipelines that leverage LLMs to generate SQL queries through structured multi-step processes. CHASE-SQL [**?**] and XiYan-SQL [**?**] both implement multi-agent frameworks in which one agent retrieves relevant schema information, another generates candidate queries, and a final agent selects the most appropriate SQL statement. CHASE-SQL employs methods such as divide-and-conquer chain-of-thought (CoT) reasoning, query-plan-based CoT reasoning, and online synthetic example generation to enhance candidate generation. In contrast, XiYan-SQL integrates supervised fine-tuning of multiple specialized generators alongside in-context learning (ICL) with skeleton-based example selection to produce high-quality and diverse SQL candidates. CHASE-SQL and XiYan-SQL rank 2nd and 4th respectively for the BIRD benchmark.

While these recent frameworks demonstrate the effectiveness of multi-agent pipelines for improving Text2SQL performance, there remains value in systematically comparing different LLMs and framework designs under a common experimental setting. In this work, we conduct an empirical evaluation of both agentless and agent-based pipelines across multiple LLMs, aiming to better understand how model choice and pipeline structure influence SQL generation quality on the BIRD-SQL Dev Set. Specifically, we evaluate not only a larger model such as Claude 3.5 Sonnet, but also smaller, potentially more economical models like GPT-4o Mini and Llama 3.1 8B. To keep the evaluation feasible, we employ a simplified version of the agent-based framework, utilizing a single generator rather than multiple specialized generators. Additionally, to assess the consistency of each model and framework combination, we run each configuration three times.

3 Methodology

In this section, we outline the methodology used for our differential analysis of LLMs in Text2SQL generation.

3.1 Model Selection

For this study, we selected the three Large Language Models (LLMs) Claude 3.5 Sonnet, GPT-4o mini, and Llama3.1 8B. These models for their widespread adoption and diversity in model size. Particularly, Claude 3.5 Sonnet has 175 billion parameters; whereas, GPT-4o mini and Llama3.1 8B have 8 billion parameters [**?**]. Notably, Llama3.1 8B is open-source, while GPT-4o Mini and Claude 3.5 Sonnet are closed-source. By comparing these models, we aim to evaluate how performance varies not only with model size but also between open-source and closed-source LLMs.

Temperature controls the degree of randomness in the text generated by large language models during inference (https://www.ibm.com/think/topics/llm-temperature). The default temperature for Claude 3.5 Sonnet is 1.0, which represents the maximum level of randomness. For GPT-4o mini, the default temperature is 0.7; however, in this study, we increase it to 1.0 to encourage greater diversity in the SQL queries generated across three runs, thereby improving the chances of obtaining a correct result. For Llama 3.1 8B, we retain the default lower temperature, as smaller models are generally more prone to hallucinations when exposed to higher levels of randomness.

3.2 Framework Selection

There are many ways that Text2SQL can be implemented. In this study, we compare the performance difference between 2 frameworks, one that utilizes a series of AI agents (Agent-based), and one that performs without AI agents (Agentless).

1. **Agentless**: In this framework, an LLM to generate the response directly. This is synonymous with the use of a chatbot. For this framework, we will provide the schemas of every table in the database and the natural language query to the LLM. Then we will run the generated SQL query and evaluate the results based on accuracy, execution time, and resource utilization.

2. **Agent-based**: In this framework, there are distinct agents that each play a different role in the generation of the response. There are 3 agents: **Relevance Agent**, **Retrieval Agent**, and **Query Agent**. The **Relevance Agent** is responsible for determining how many tables are relevant to a natural language query. The **Schema Agent** will be responsible for retrieving the schemas of the relevant tables from the database given the natural language query. The **Query Agent** will be responsible for generating the SQL query given the relevant schemas and the natural language query. Then we will run the generated SQL query and evaluate the results based on the same criteria as the Agentless framework. The agent-based pipeline can be seen in **??**

Figure 1: Agent-based Text2SQL Pipeline

The main difference between Agentless and Agent-based framework is that during the Text2SQL generation, the schemas of all tables are used for Agentless, whereas only the schemas of relevant tables are used for Agent-based. The comparison between these frameworks can provide insight into the trade-offs between simplicity and modularity,

particularly in terms of scalability and accuracy – considering Agentless make less calls to the LLM than Agent-based. Specifically, we aim to explore whether introducing specialized agents leads to more accurate query generation, or if the added complexity results in diminishing returns compared to a simpler, monolithic approach. This evaluation can help guide future development of LLM-powered database interfaces.

3.3 Benchmark Dataset Selection

The SQL databases that we plan to use are the the BIRD-SQL Dev Set and the Chinook sample database, which is a sample database provided by SQLite. For each of the tables in the Chinook database, we will create the Schemas, which will be used as the context passed to the LLMs. The BIRD-SQL Dev Set includes the Schemas of the tables in its database. Additionally, we will determine 15 natural language queries for BIRD-SQL and Chinook, their corresponding SQL queries, and the results of executing the SQL queries. Using that benchmark dataset as ground truth, we will perform a differential analysis of the performance of 3 LLMs for the 2 frameworks. We will analyze the ability of the LLMs to generate correct SQL queries, as well as the differences in performance between the LLMs and frameworks.

We will test every Model+Framework combination on every natural language query 3 times. This will allow us to determine the consistency of the Model+Framework in generating SQL queries.

4 Results

For this update, we have run the 'Agentless' and 'Agent-based' framework with GPT-4o on BIRD-SQL and Chinook. BIRD-SQL benchmark contains 7 simple, 5 moderate, and 3 challenging natural language queries – categories provided by BIRD-SQL. Chinook also contains 15 natural language queries, but their difficulty isn't categorized as they were manually created by us.

4.1 Correctness and Consistency

4.1.1 BIRD-SQL

The results from the study on the BIRD-SQL benchmark can be seen in Figure **??**. Firstly, the results demonstrate the Text2SQL is a challenging task. For some questions, including a simple one, neither frameworks were able to generate a correct SQL query even once. Secondly, the results reveal that most of the time, the Agent-based system is more consistently correct, shown by the rows with ID 1, 6, and 15. Agentless was able to generate a solution when Agent-based failed only once, shown by the row with ID 9.

Figure 2: GPT-4o mini results on BIRD-SQL

Agents are employed to retrieve relevant schemas for the Agent-based system. These relevant schemas are used during the Text2SQL generation. However, for Agentless system, all schemas are used during the Text2SQL generation. This means that irrelevant schemas could be passed as context to the LLM, resulting in incorrect SQL queries. This is consistent with the findings from Figure **??**.

4.1.2 Chinook

The results from the study on the Chinook benchmark can be seen in Figure **??**. Firstly, the model was able to generate SQL queries for almost all of the natural language queries, which was not the case for the BIRD-SQL benchmark. This could be due to our lack of expertise in SQL, resulting in simple queries in our benchmark. Due to this, for the final report, we are considering evaluating the Model+Framework combinations on a larger BIRD-SQL benchmark, and omitting the Chinook benchmark. Secondly, the perfoormance of both frameworks was very similar, with discrepencies in only 3 queries, shown by the rows with ID 1, 13, and 14. It is interesting to note that Agentless technically performs slightly better than Agent-based, which contradicts the results from the BIRD-SQL benchmark. However, it isn't a significant enough performance difference to draw a difinitive conclusion, especially given the aforementioned simplicity of the benchmark.

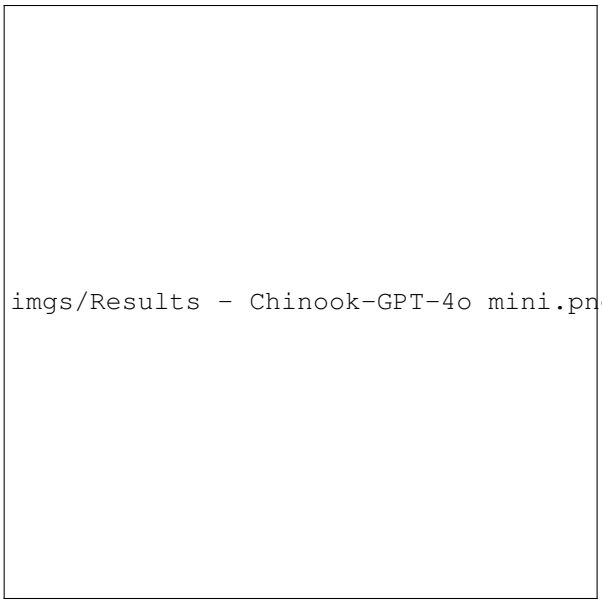

Figure 3: GPT-4o mini results on Chinook

5 Conclusion

6 Additional Notes

We will continue running this study on Llama 3 8B and another LLM, which is still to be determined. As mentioned earlier, we also plan to omit the Chinook benchmark moving forward and expand the size of the BIRD-SQL benchmark. This change provides several benefits. First, manually creating natural language queries and their corresponding ground truth answers is prone to errors, especially given our limited SQL expertise. Second, BIRD-SQL offers natural language queries paired with corresponding SQL queries, enabling us to scale our system to a larger benchmark since we no longer have to manually determine the ground truth answers. Finally, BIRD is a widely used benchmark in Text-to-SQL research and its diversity in schema and query complexity also allows for more robust evaluation of framework and model performance.